DEPARTMENT OF INFORMATION ENGINEERING AND COMPUTER SCIENCE
UNIVERSITY OF TRENTO

SYNAPSE HEA

Hospital Management System

(HEA)

COMPLETE SOFTWARE ENGINEERING DOCUMENTATION

Java 21 | Spring Boot | React | TypeScript

AUTHOR

Mohammad Nomaan Munshi

Software Engineering [146148] | DISI | 2025/2026

Project history

Synapse HEA was redesigned, implemented, and documented independently by Mohammad Nomaan Munshi. The current edition replaces the early MERN-style prototype with a Java 21, Spring Boot, React, TypeScript, MySQL, Redis, Docker, and Nginx architecture.

01 Authorship and Project Evolution

Independent design, implementation, and consolidated documentation

Edition	Contribution and ownership
Academic requirements and early prototype	Independent analysis and development by Mohammad Nomaan Munshi.
Independent Java full-stack rebuild	Independent re-engineering by Mohammad Nomaan Munshi.
Independent consolidated documentation	Rewritten as one consistent document using the old deliverables as historical input and the Java rebuild as the current baseline.

Honest project description

Synapse HEA is an independently redesigned Java and Spring Boot healthcare-management system created and documented by Mohammad Nomaan Munshi.

Why this edition exists

- The old documents used inconsistent product names and scope.
- Some requirements described features that were not implemented.
- D2 repeated diagrams and used mixed identifiers for the same use cases.
- D3 documented a Node.js and Express implementation that is no longer the current architecture.
- The current edition creates one traceable baseline for requirements, design, implementation, testing, and deployment.

02 Revision History

Document control for the consolidated edition

Version	Date	Author	Summary
1.0	2025/2026	Mohammad Nomaan Munshi	University D1, D2, and D3 submissions.
2.0	August 2026	Mohammad Nomaan Munshi	Consolidated and corrected Java full-stack rebuild documentation.

Document status

Item	Status
Requirements baseline	Complete
Use-case model	Complete
Architecture model	Complete
Representative API catalogue	Complete
Database model	Complete
Test strategy	Complete
Production compliance certification	Not claimed
Source-code audit against the private repository	Not performed in this document

> **Important limitation**
> This is professional project documentation for an academic and portfolio system. It is not a certification that the application is ready for real clinical use.

03 Executive Summary

What Synapse HEA is and what the rebuild changes

Synapse HEA is a web-based healthcare management system focused on secure access, appointment coordination, medical records, prescriptions, billing records, document handling, notifications, and administration. The rebuild uses a modular monolith so that the code remains understandable for one developer while still separating business domains cleanly.

Area	Version 2.0 decision
Frontend	React and TypeScript built with Vite, served by Nginx.
Backend	Java 21 and Spring Boot with REST APIs.
Security	Spring Security, short-lived JWT access tokens, rotating refresh tokens in HttpOnly cookies, BCrypt password hashing, and role-based access control.
Persistence	MySQL with JPA/Hibernate, Flyway migrations, validation, transactions, UUID identifiers, and optimistic locking where needed.
Real-time behavior	Server-Sent Events for notifications.
Operations	Docker, Nginx reverse proxy, Actuator, Prometheus-compatible metrics, and structured audit events.

> **Design goal**
> Use the simplest architecture that is credible, secure, and maintainable for the project size. Microservices were deliberately avoided because they would add operational complexity without a clear benefit.

04 Document Map

How the complete documentation is organised

Part	Topic	Contents
Part I	Project definition	Background, objectives, scope, stakeholders, and product boundaries.
Part II	Requirements	Functional requirements, non-functional requirements, and traceability.
Part III	Use cases and behavior	Actor goals, main scenarios, alternative flows, and activity diagrams.
Part IV	Architecture and data	Application layers, modules, deployment, security, and database design.

Part	Topic	Contents
Part V	Implementation interface	API catalogue, frontend and backend structure, and sequence diagrams.
Part VI	Quality and operations	Testing, CI/CD, monitoring, errors, privacy, limitations, and roadmap.
Appendices	Reference material	Status matrix, example payloads, repository structure, and review checklist.

Reading path
Recruiters can read the executive summary, architecture, security, testing, and migration sections first. Developers can use the requirements, use cases, API catalogue, ERD, and data dictionary as the working baseline.

05 Project Background

Why the system was created

Healthcare organisations coordinate people, schedules, records, documents, and administrative work. When these processes are split across paper forms or disconnected tools, users face avoidable waiting, duplicate data, scheduling conflicts, and poor visibility.

- Patients need a straightforward way to find doctors, book appointments, view history, and receive updates.
- Doctors need reliable schedules, patient context, and controlled ways to add clinical records and prescriptions.
- Administrators need user management, operational dashboards, reports, and an audit trail.
- Developers need a maintainable system with clear modules, schema migrations, observable failures, and repeatable deployment.

Project position
Synapse HEA is a learning and portfolio system that demonstrates software engineering practices. It does not replace a certified hospital information system.

06 Problem Statement

The specific problems addressed by the system

Problem	Impact	System response
Manual or fragmented appointment handling	Long waits, duplicate bookings, and poor communication.	Central appointment lifecycle with availability checks and status updates.
Weak identity and access controls	Sensitive information may be exposed to the wrong user.	Authentication, RBAC, short-lived access tokens, refresh rotation, and audit logging.
Scattered medical information	Doctors and patients may not see the same history.	Central medical records, prescriptions, documents, and controlled access.
No timely update channel	Users repeatedly refresh the system or miss changes.	Persistent notifications plus an authenticated SSE stream.
Unclear operational health	Failures are difficult to diagnose.	Health endpoints, metrics, structured errors, and audit events.

07 Objectives and Success Criteria

Measurable outcomes for Version 2.0

ID	Objective
O1	Provide secure role-based access for patients, doctors, and administrators.
O2	Allow patients to discover doctors and manage appointments without schedule conflicts.
O3	Support controlled access to medical records, prescriptions, billing records, and documents.
O4	Deliver clear dashboards and notifications for each role.
O5	Maintain consistent data through transactions, database constraints, and explicit status transitions.

ID	Objective
O6	Keep the codebase maintainable through domain modules, tests, migrations, and API documentation.
O7	Make deployment reproducible through Docker and Nginx.
O8	Expose health and metrics information for troubleshooting and monitoring.

Success criteria

- A patient cannot create overlapping appointments for the same doctor and time.
- A user cannot access resources outside the permissions of their role.
- Password values are never stored in plain text.
- Refresh tokens can be rotated and revoked.
- Important state changes create both a notification and an audit record.
- Database changes are versioned and repeatable.
- Core APIs have automated tests for success and failure paths.

08 Scope and Boundaries

What Version 2.0 includes and excludes

In scope	Out of scope
User registration, login, refresh, logout, and role-based access	AI diagnosis or treatment recommendations
Patient, doctor, and administrator profiles	Medical device integration
Doctor discovery and availability	National health database integration
Appointment booking, rescheduling, cancellation, confirmation, and rejection	Insurance claim adjudication
Medical records and prescriptions	Ambulance tracking
Billing records without direct payment processing	Telemedicine video calls
Document upload metadata and controlled access	Emergency triage automation
Notifications, SSE updates, dashboards, audit logs, metrics	Production certification for real patient care

Boundary decision

The old resource-management and urgent-triage requirements are not carried into the current baseline because they do not match the verified project modules described for the Java rebuild.

09 Stakeholders and Roles

People and systems affected by Synapse HEA

Stakeholder	Primary interest	Main responsibilities
Patient	Easy and secure access to healthcare services.	Manage profile, appointments, records, documents, and notifications.
Doctor	Accurate schedule and patient context.	Manage availability, appointment decisions, records, and prescriptions.
Administrator	Operational control and accountability.	Manage users, review metrics, reports, and audit logs.
Developer / operator	Reliable and maintainable software.	Deploy, monitor, migrate, test, and troubleshoot the system.
Email provider	Reliable message delivery.	Transmit transactional messages.
Database service	Durable and consistent data.	Store application data and enforce constraints.

Role simplification

Nurse and receptionist roles from the early documents are not separate roles in the current baseline. Adding them later should be done through explicit permissions rather than copying administrator access.

10 System Context

External actors and system dependencies

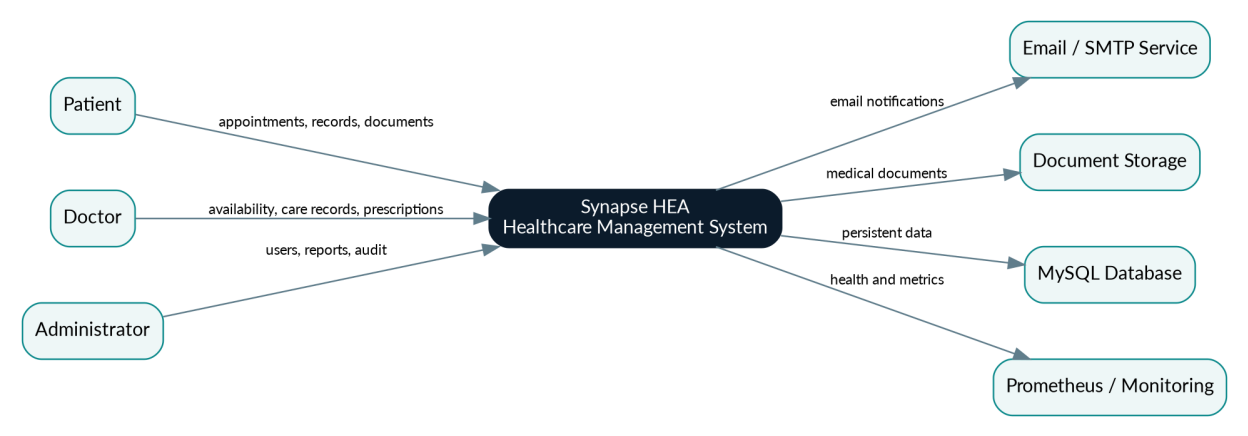


Figure 1. System context for Synapse HEA Version 2.0.

Context rule

The application owns healthcare workflow data but depends on external infrastructure for email delivery, document storage, database persistence, and metrics collection.

11 Product Overview

Role-specific experience across one shared platform

Role	Primary screens	Typical outcome
Patient	Dashboard, doctor search, appointment form, history, records, documents, notifications.	Book care and stay informed.
Doctor	Dashboard, availability, appointment queue, patient history, record editor, prescription editor.	Manage workload and document care.
Administrator	Dashboard, users, appointments, audit logs, reports, health status.	Operate and supervise the platform.

Shared behavior

- All protected screens depend on authenticated API requests.
- The backend, not the frontend, enforces role permissions and business rules.
- Every significant state change has a clear status, timestamp, and actor.
- Errors are returned in a consistent problem-details format.
- Notifications are persisted before they are streamed in real time.

12 Functional Requirements 1-8

Normalised requirement catalogue for the Java rebuild

ID	Requirement	Definition	Priority
FR1	User registration	Allow a new patient or authorised staff user to create an account with validated details.	Core
FR2	Login	Authenticate an active user with email and password.	Core
FR3	Password management	Support password change and secure reset flows.	Core
FR4	Profile management	Allow users to view and update permitted profile fields.	Core

ID	Requirement	Definition	Priority
FR5	Doctor search	Find doctors by name, speciality, or availability.	Core
FR6	Doctor availability	Create, update, and query doctor availability windows.	Core
FR7	Appointment booking	Create an appointment only when the selected slot remains available.	Core
FR8	Appointment modification	Reschedule or cancel an appointment under status and time rules.	Core

13 Functional Requirements 9-16

Normalised requirement catalogue for the Java rebuild

ID	Requirement	Definition	Priority
FR9	Appointment status workflow	Support pending, confirmed, rejected, rescheduled, cancelled, and completed states.	Core
FR10	Medical record access	Allow authorised users to view patient records.	Core
FR11	Medical record maintenance	Allow doctors to create and update clinical notes linked to patient and appointment.	Core
FR12	Prescription management	Allow doctors to create and review prescriptions linked to a medical record.	Core
FR13	Billing records	Maintain invoice or charge records without processing external payments.	Core
FR14	Document handling	Upload, list, download, and control access to patient documents.	Core
FR15	Notification inbox	Store and display user notifications and unread status.	Core
FR16	Real-time notifications	Stream new notification events to authenticated clients using SSE.	Core

14 Functional Requirements 17-24

Normalised requirement catalogue for the Java rebuild

ID	Requirement	Definition	Priority
FR17	Role-based access control	Enforce patient, doctor, and administrator permissions on every protected endpoint.	Core
FR18	Token lifecycle	Issue short-lived access tokens and rotate revocable refresh tokens.	Core
FR19	Audit logging	Record important security and business events with actor and timestamp.	Core
FR20	Admin user management	Allow administrators to list, activate, suspend, and review users.	Core
FR21	Doctor schedule management	Allow doctors to manage availability and avoid conflicting slots.	Core
FR22	Appointment history	Show role-appropriate historical appointments with status.	Core

ID	Requirement	Definition	Priority
FR23	Patient dashboard	Aggregate upcoming appointments, recent records, and notifications.	Core
FR24	Doctor dashboard	Aggregate appointment requests, schedule, and patient context.	Core

15 Functional Requirements 25-32

Normalised requirement catalogue for the Java rebuild

ID	Requirement	Definition	Priority
FR25	Admin dashboard	Provide system counts, recent activity, and health indicators.	Core
FR26	Reporting	Generate filtered operational summaries from authorised data.	Extended
FR27	API documentation	Expose current OpenAPI documentation for supported endpoints.	Core
FR28	Health monitoring	Expose health, readiness, and metrics endpoints.	Core
FR29	Error handling	Return stable error codes, safe messages, and correlation details.	Core
FR30	Schema migration	Apply ordered Flyway migrations during deployment.	Core
FR31	Backup readiness	Support database and document backup procedures.	Operational
FR32	Logout and revocation	Revoke refresh credentials and clear the authentication cookie.	Core

16 Non-Functional Requirements 1-8

Quality constraints and operational expectations

ID	Quality	Requirement
NFR1	Security	Protected endpoints require authenticated and authorised access.
NFR2	Privacy	Only the minimum role-appropriate data is returned.
NFR3	Password protection	Passwords are hashed with BCrypt and never logged.
NFR4	Transport security	Production traffic uses HTTPS and secure cookies.
NFR5	Response time	Normal interactive API requests should complete within 3 seconds under expected academic-project load.
NFR6	Concurrency	Appointment creation prevents double booking under concurrent requests.
NFR7	Availability	The system should recover cleanly from process restarts and dependency failures.
NFR8	Error resilience	A failed request does not corrupt committed data or crash unrelated functions.

Interpretation

These are engineering targets for the project. They are not production service-level agreements.

17 Non-Functional Requirements 9-15

Quality constraints and operational expectations

ID	Quality	Requirement
NFR9	Usability	Role-specific screens use clear labels, status chips, and actionable errors.
NFR10	Accessibility	Keyboard use, semantic labels, focus visibility, and responsive layouts are supported.
NFR11	Data consistency	Transactions and constraints maintain valid relationships and statuses.
NFR12	Maintainability	Modules, DTOs, services, repositories, tests, and migrations follow consistent conventions.
NFR13	Observability	Health, metrics, audit events, and structured logs support diagnosis.
NFR14	Scalability	The modular monolith can scale vertically and through multiple stateless backend instances with shared stores.
NFR15	Recoverability	Database and document storage can be backed up and restored using documented procedures.

Interpretation

These are engineering targets for the project. They are not production service-level agreements.

18 Requirements Traceability

From objectives to requirements and verification

Objective	Requirements	Verification evidence
O1	FR1-FR4, FR17-FR18, FR32	Authentication and authorisation tests
O2	FR5-FR9, FR21-FR24	Appointment integration tests and conflict tests
O3	FR10-FR16	Medical, document, billing, and notification tests
O4	FR23-FR26	Dashboard and reporting tests
O5	FR7-FR9, FR19, FR30	Transaction, status, audit, and migration tests
O6	FR27, NFR12	OpenAPI review and code-structure review
O7	FR30-FR31	Container startup and migration checks
O8	FR28-FR29, NFR13	Health, metrics, and error-contract checks

Traceability rule

A requirement is not considered complete only because a screen exists. It needs backend enforcement and a repeatable verification method.

19 Use-Case Overview

Primary user goals and system responsibilities

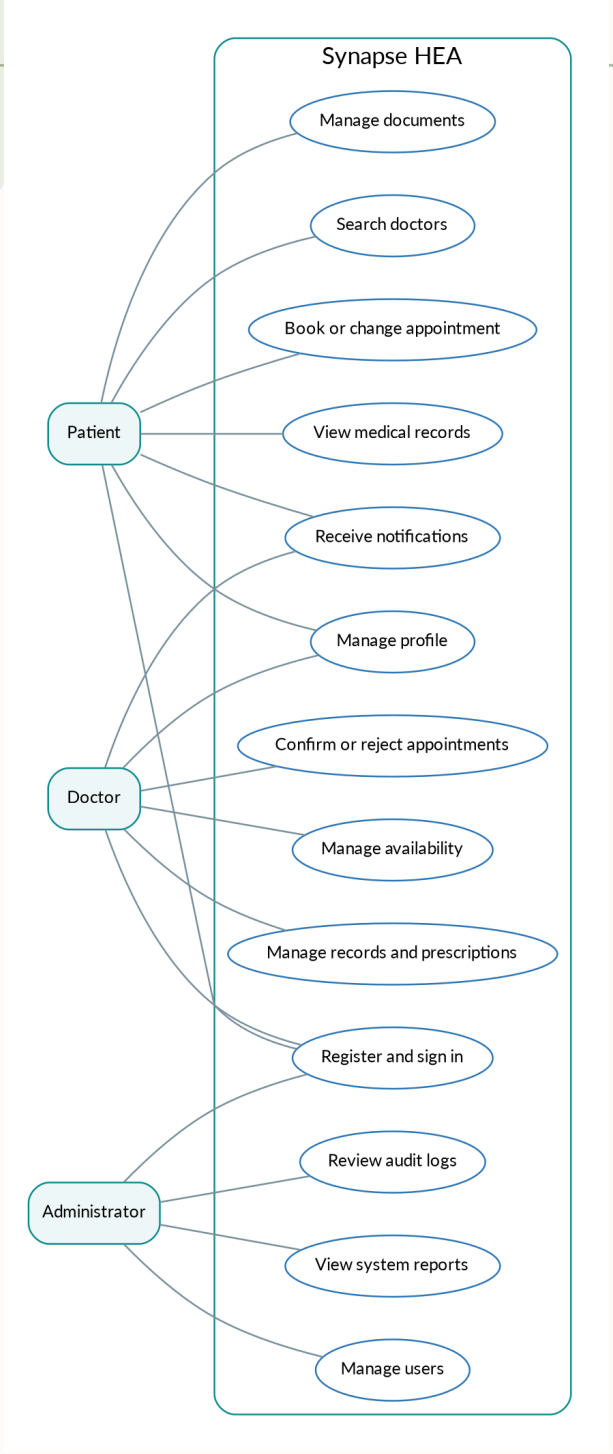


Figure 2. Consolidated use-case overview.

Model decision
Shared functions such as authentication and notifications are not duplicated as separate diagrams for every role. This keeps the model readable and removes the repetition present in the old D2.

UC01 Register Account
Create a validated user account and role profile.

Field	Definition
Primary actor	New user
Supporting actors	System services
Preconditions	The email is not already registered.
Trigger	The user submits the registration form.

Field	Definition
Success result	A user and profile are stored and a safe success response is returned.
Minimal guarantee	No partial account remains after failure.

Main success scenario

1. User enters required identity and contact data.
2. Frontend performs basic validation and submits the request.
3. Backend validates the request and normalises the email.
4. System checks uniqueness.
5. Password is hashed with BCrypt.
6. User and role profile are created in one transaction.
7. An audit event is recorded.
8. The system returns a successful response.

Alternative and error flows

- Invalid input returns field-level errors.
- Duplicate email returns a conflict response.
- Database failure rolls back the transaction.

> **Business rules**
> Registration never accepts a client-supplied administrator role.

UC02 Login
Authenticate a registered user and create a secure session.

Field	Definition
Primary actor	Registered user
Supporting actors	System services
Preconditions	The account exists and is active.
Trigger	The user submits email and password.
Success result	Access credentials and user summary are returned.
Minimal guarantee	No token is issued for invalid or suspended users.

Main success scenario

9. User submits credentials.
10. Backend loads the user by normalised email.
11. BCrypt verifies the password.
12. Account status and role are checked.
13. A short-lived access token is created.
14. A refresh token is created, hashed, and stored.
15. Refresh token is set in an HttpOnly SameSite cookie.
16. Login is audited and the user is redirected.

Alternative and error flows

- Invalid credentials return the same generic message.
- Suspended users are denied.
- Rate limiting can delay repeated failures.

> **Business rules**
> The raw refresh token is never stored in the database.

UC03 Refresh or Logout
Rotate session credentials or revoke the active refresh token.

Field	Definition
Primary actor	Authenticated user

Field	Definition
Supporting actors	System services
Preconditions	A refresh cookie is present.
Trigger	Access token expires or the user selects logout.
Success result	A rotated session is issued, or the current session is revoked.
Minimal guarantee	A reused, expired, or revoked token cannot create a new session.

Main success scenario

17. Browser sends the refresh cookie.
18. Backend hashes the token and finds the active record.
19. User status is checked.
20. Old token is revoked.
21. A new access token and refresh token are created.
22. New refresh cookie is returned.
23. For logout, the token is revoked and the cookie is cleared.

Alternative and error flows

- Missing or invalid cookie returns unauthorised.
- Detected token reuse can revoke the related token family.

> **Business rules**
> Refresh rotation is performed atomically.

UC04 Manage Profile
View and update role-appropriate personal information.

Field	Definition
Primary actor	Patient or doctor
Supporting actors	System services
Preconditions	The user is authenticated.
Trigger	The user saves profile changes.
Success result	Allowed fields are updated and audited.
Minimal guarantee	Protected fields such as role and account status are unchanged.

Main success scenario

24. User opens the profile page.
25. System loads the current user summary.
26. User changes allowed fields.
27. Backend validates the request.
28. Service authorises each field.
29. Profile is updated with a new modification timestamp.
30. System returns the updated profile.

Alternative and error flows

- Invalid phone, date, or required fields return validation errors.
- Concurrent edit returns a conflict when version checks fail.

> **Business rules**
> Only administrators may change another user account status.

UC05 Search Doctors
Find a suitable doctor using trusted server-side filters.

Field	Definition
Primary actor	Patient

Field	Definition
Supporting actors	System services
Preconditions	The patient is authenticated.
Trigger	The patient enters search criteria.
Success result	A paginated list of matching active doctors is returned.
Minimal guarantee	No sensitive staff data is exposed.

Main success scenario

31. Patient enters name, speciality, date, or availability filter.
32. Frontend sends query parameters.
33. Backend validates pagination and filter values.
34. Repository performs indexed search.
35. Service maps results to public doctor summaries.
36. UI displays matching doctors and next available times.

Alternative and error flows

- No results returns an empty list with filter guidance.
- Invalid sort field returns a validation error.

> **Business rules**
> Search never returns private contact or account-security fields.

UC06 Manage Doctor Availability
Create and maintain valid appointment windows.

Field	Definition
Primary actor	Doctor
Supporting actors	System services
Preconditions	The doctor account is active.
Trigger	The doctor adds or edits availability.
Success result	A non-overlapping availability window is stored.
Minimal guarantee	Existing availability remains valid after a failed change.

Main success scenario

37. Doctor selects date and time range.
38. Backend verifies role and ownership.
39. System validates duration and future date rules.
40. System checks overlap with existing windows.
41. Availability is created or updated.
42. Audit event is written.
43. Updated schedule is returned.

Alternative and error flows

- Overlapping windows return conflict.
- Window with confirmed appointments cannot be deleted without a defined replacement flow.

> **Business rules**
> Availability does not itself create an appointment.

UC07 Book Appointment
Create a conflict-free appointment for a patient and doctor.

Field	Definition
Primary actor	Patient

Field	Definition
Supporting actors	System services
Preconditions	Patient and doctor are active and the selected time is in an available window.
Trigger	Patient confirms a selected slot.
Success result	A pending appointment is stored and both parties are notified.
Minimal guarantee	No duplicate booking is committed.

Main success scenario

44. Patient chooses doctor, date, and slot.
45. System validates the request and ownership.
46. Backend starts a transaction.
47. Availability and overlapping appointments are checked with locking or a unique constraint.
48. Appointment is created with PENDING status.
49. Audit and notification records are created.
50. Transaction commits.
51. SSE event is published and the appointment is returned.

Alternative and error flows

- Slot no longer available returns conflict and refreshed slots.
- Invalid reason or date returns validation error.

> **Business rules**
> The final availability check occurs on the server inside the transaction.

UC08 Reschedule or Cancel Appointment
Change an appointment while preserving history and consistency.

Field	Definition
Primary actor	Patient or doctor
Supporting actors	System services
Preconditions	The appointment exists and the actor is authorised.
Trigger	The actor selects reschedule or cancel.
Success result	The appointment status and schedule are updated, audited, and notified.
Minimal guarantee	The old state remains if the operation fails.

Main success scenario

52. Actor opens an appointment.
53. System loads the current version and permissions.
54. Requested transition is validated.
55. For reschedule, the new slot is checked again.
56. Appointment is updated with version control.
57. Status history and audit event are recorded.
58. Notification is created and streamed.

Alternative and error flows

- Invalid transition returns conflict.
- Concurrent change returns version conflict.
- Past or completed appointment cannot be rescheduled.

> **Business rules**
> Status transitions follow an explicit state machine.

UC09 Doctor Manages Appointment Requests
Confirm, reject, reschedule, or complete assigned appointments.

Field	Definition
Primary actor	Doctor
Supporting actors	System services
Preconditions	The doctor owns the appointment.
Trigger	A doctor opens the appointment queue.
Success result	A valid decision is recorded and the patient is informed.
Minimal guarantee	A doctor cannot modify another doctor appointment.

Main success scenario

59. Doctor loads pending and upcoming appointments.
60. Doctor selects an appointment.
61. System returns role-safe patient context.
62. Doctor chooses a valid action.
63. Backend validates ownership and transition.
64. Appointment is updated and audited.
65. Patient notification is created.

Alternative and error flows

- Rejected appointment requires a reason.
- Completion may require a linked medical record according to configured rules.

> **Business rules**
> Ownership is checked using the authenticated doctor identity, not a client-supplied doctor id.

UC10 Manage Medical Records
Create and review patient records with controlled access.

Field	Definition
Primary actor	Doctor
Supporting actors	Patient, administrator with limited operational access
Preconditions	The doctor is authorised to access the patient context.
Trigger	Doctor opens the medical record editor.
Success result	A record is stored with author, patient, and timestamp.
Minimal guarantee	Unauthorised users receive no record content.

Main success scenario

66. Doctor opens an appointment or patient context.
67. System verifies access relationship.
68. Existing records are loaded in reverse chronological order.
69. Doctor adds diagnosis, notes, and follow-up details.
70. Backend validates and stores the record.
71. Audit event is written.
72. Patient can later view the permitted record.

Alternative and error flows

- Missing relationship returns forbidden.
- Invalid content returns validation errors.

> **Business rules**
> Records are not hard-deleted through normal user flows.

UC11 Manage Prescriptions
Attach clear medication instructions to a medical record.

Field	Definition
Primary actor	Doctor
Supporting actors	System services
Preconditions	A medical record exists and belongs to the doctor workflow.
Trigger	Doctor adds a prescription.
Success result	Prescription items are stored and visible to the patient.
Minimal guarantee	No prescription is created without a linked record.

Main success scenario

73. Doctor opens a medical record.
74. Doctor enters medication, dosage, frequency, and duration.
75. Backend validates required fields.
76. Prescription is linked to the record and author.
77. Audit event is written.
78. Patient notification can be created.

Alternative and error flows

- Invalid or missing fields return validation errors.
- Attempt to modify another doctor prescription is denied.

> **Business rules**
> The system stores instructions but does not provide automated prescribing advice.

UC12 Manage Billing Records
Track charges and payment status without processing money.

Field	Definition
Primary actor	Administrator
Supporting actors	Patient
Preconditions	A billable appointment or service exists.
Trigger	Administrator creates or updates a billing record.
Success result	Amount, currency, status, and reference are stored.
Minimal guarantee	Clinical records are unaffected by billing failure.

Main success scenario

79. Administrator opens the billing module.
80. System loads billable appointments.
81. Administrator enters amount and description.
82. Backend validates money values and permissions.
83. Billing record is stored and audited.
84. Patient can view the allowed billing summary.

Alternative and error flows

- Negative or malformed amount is rejected.
- External payment status can be updated only through a future verified integration.

> **Business rules**
> Version 2.0 records billing information but does not operate a payment gateway.

UC13 Manage Patient Documents
Upload and retrieve documents through controlled metadata and storage references.

Field	Definition
Primary actor	Patient or authorised doctor

Field	Definition
Supporting actors	System services
Preconditions	The user is authenticated and has access to the patient context.
Trigger	The user selects a document for upload.
Success result	The file is stored and a metadata record is created.
Minimal guarantee	Unsupported or oversized files are rejected before persistence.

Main success scenario

85. User selects a file and document category.
86. Frontend submits multipart data.
87. Backend checks size, type, and access.
88. File is stored using a generated storage key.
89. Metadata is saved in the database.
90. Audit event is recorded.
91. Document list is refreshed.

Alternative and error flows

- Unsupported type returns validation error.
- Storage failure removes incomplete metadata.

> **Business rules**
> Original filenames are not trusted as storage paths.

UC14 Receive Notifications
Persist and deliver appointment and account updates.

Field	Definition
Primary actor	Authenticated user
Supporting actors	System services
Preconditions	The user is signed in.
Trigger	A business event requires user notification.
Success result	Notification appears in the inbox and, when connected, arrives through SSE.
Minimal guarantee	The persistent notification remains available even if real-time delivery fails.

Main success scenario

92. Business service creates a notification record.
93. Transaction commits.
94. Notification event is published.
95. SSE service sends the event to the authenticated user connection.
96. UI displays the new notification.
97. User can mark it as read.

Alternative and error flows

- Disconnected client receives the notification on the next inbox request.
- Expired session closes the stream.

> **Business rules**
> SSE is an optimisation, not the source of truth.

UC15 Administer Users and Audit
Operate accounts and inspect significant system activity.

Field	Definition
Primary actor	Administrator

Field	Definition
Supporting actors	System services
Preconditions	Administrator is authenticated with active privileges.
Trigger	Administrator opens user or audit management.
Success result	Authorised change or filtered audit query completes.
Minimal guarantee	Administrator actions are themselves audited.

Main success scenario

98. Administrator searches users or audit events.
99. Backend validates filters and pagination.
100. System returns role-safe results.
101. Administrator performs an allowed account action.
102. Service validates the transition.
103. Account state changes.
104. Audit event records actor, target, action, and time.

Alternative and error flows

- Administrator cannot deactivate their own only active admin account under configured safety rules.
- Invalid state transition returns conflict.

> **Business rules**
> Audit records are append-only through normal application APIs.

35 Registration Activity

Corrected activity flow for account creation

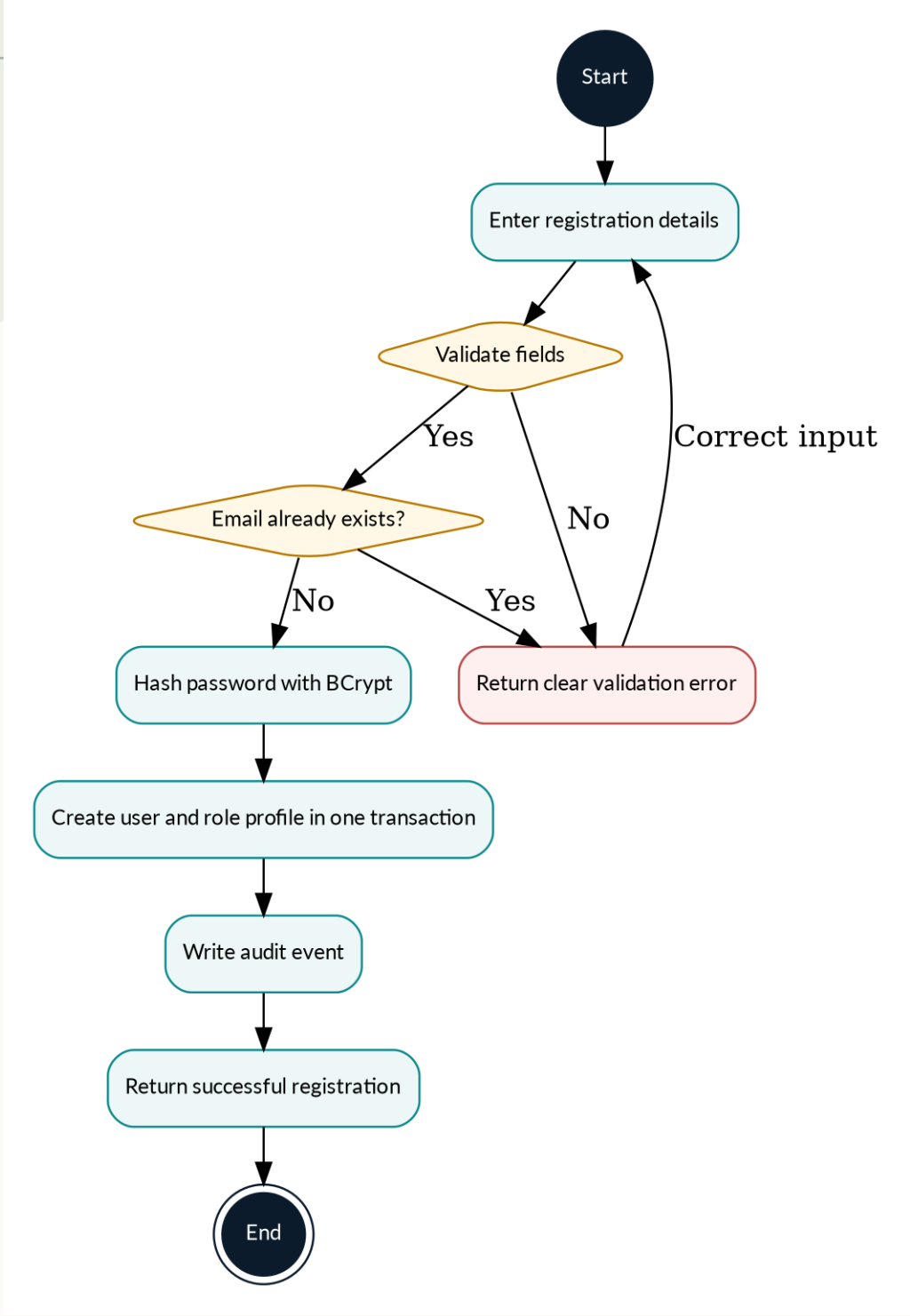


Figure 3. Registration activity with validation, uniqueness, transaction, and audit.

Correction from old diagrams
Validation errors return to input. Successful and failed paths are not incorrectly merged before account creation.

36 Appointment Lifecycle Activity

Corrected booking and decision flow

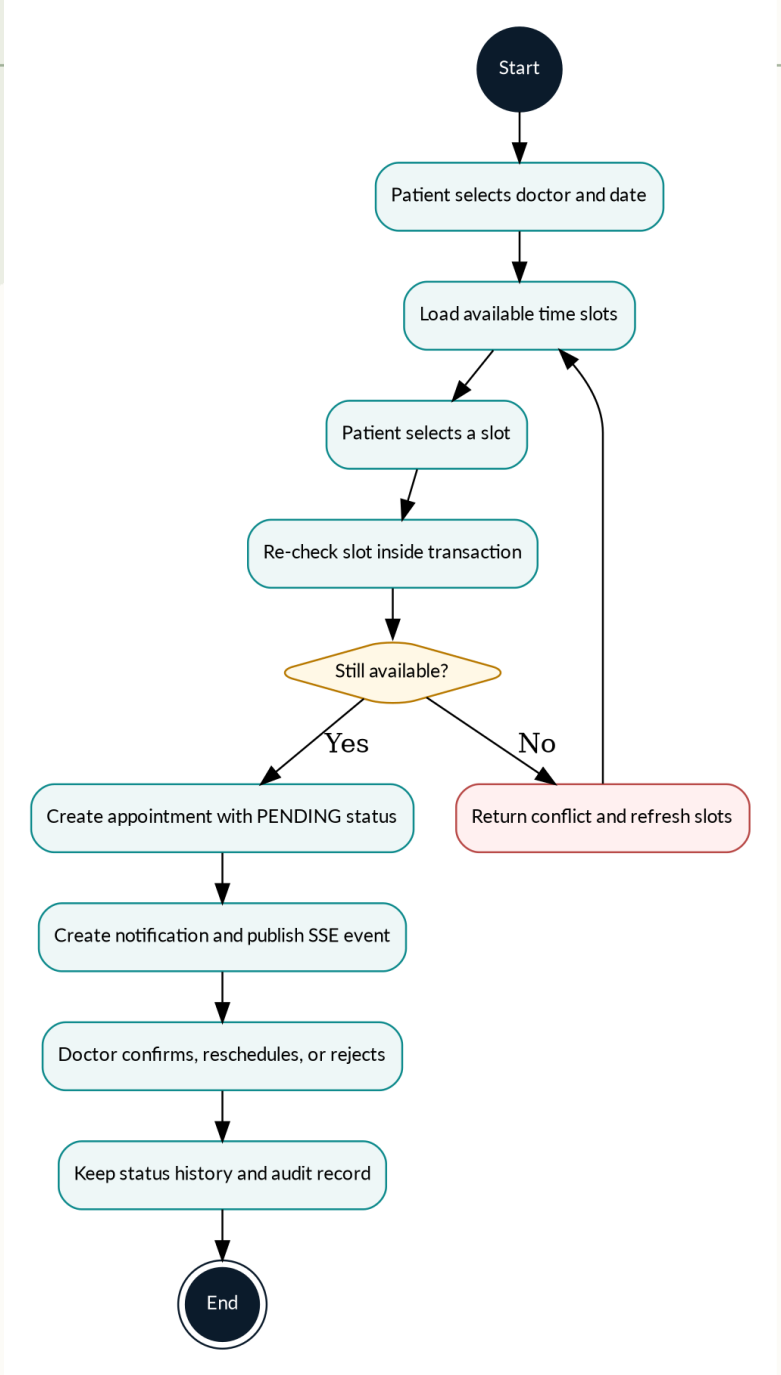


Figure 4. Appointment lifecycle with final conflict check and notifications.

Consistency rule
Availability shown in the UI is advisory. The server performs the authoritative check at commit time.

37 Architecture Overview

Multi-tier modular monolith for the Java rebuild

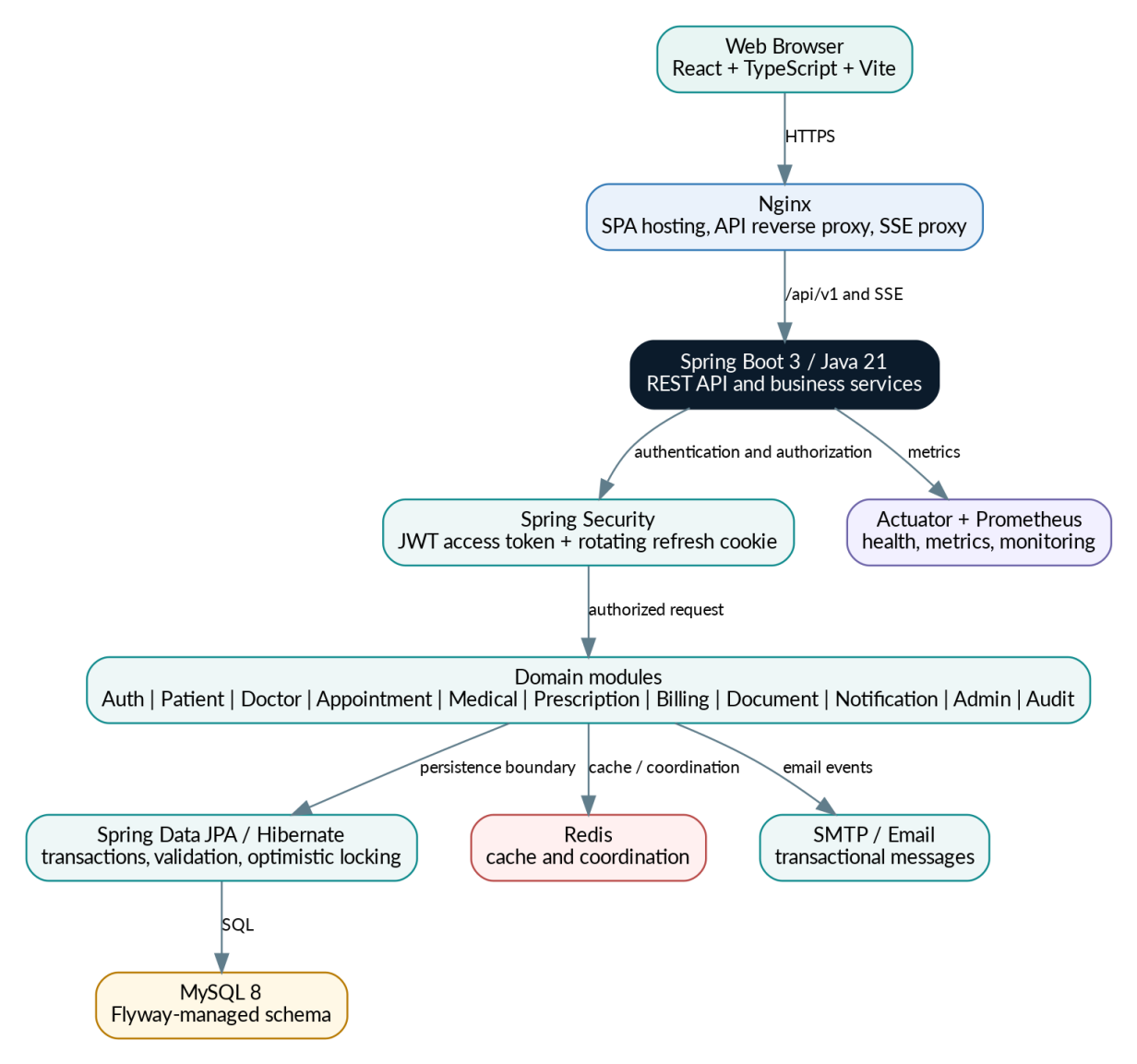


Figure 5. Version 2.0 application architecture.

Why a modular monolith

It keeps deployment simple while preserving clean domain boundaries. This is more appropriate than microservices for the current team size and project maturity.

38 Component Model

Domain modules and their main dependencies

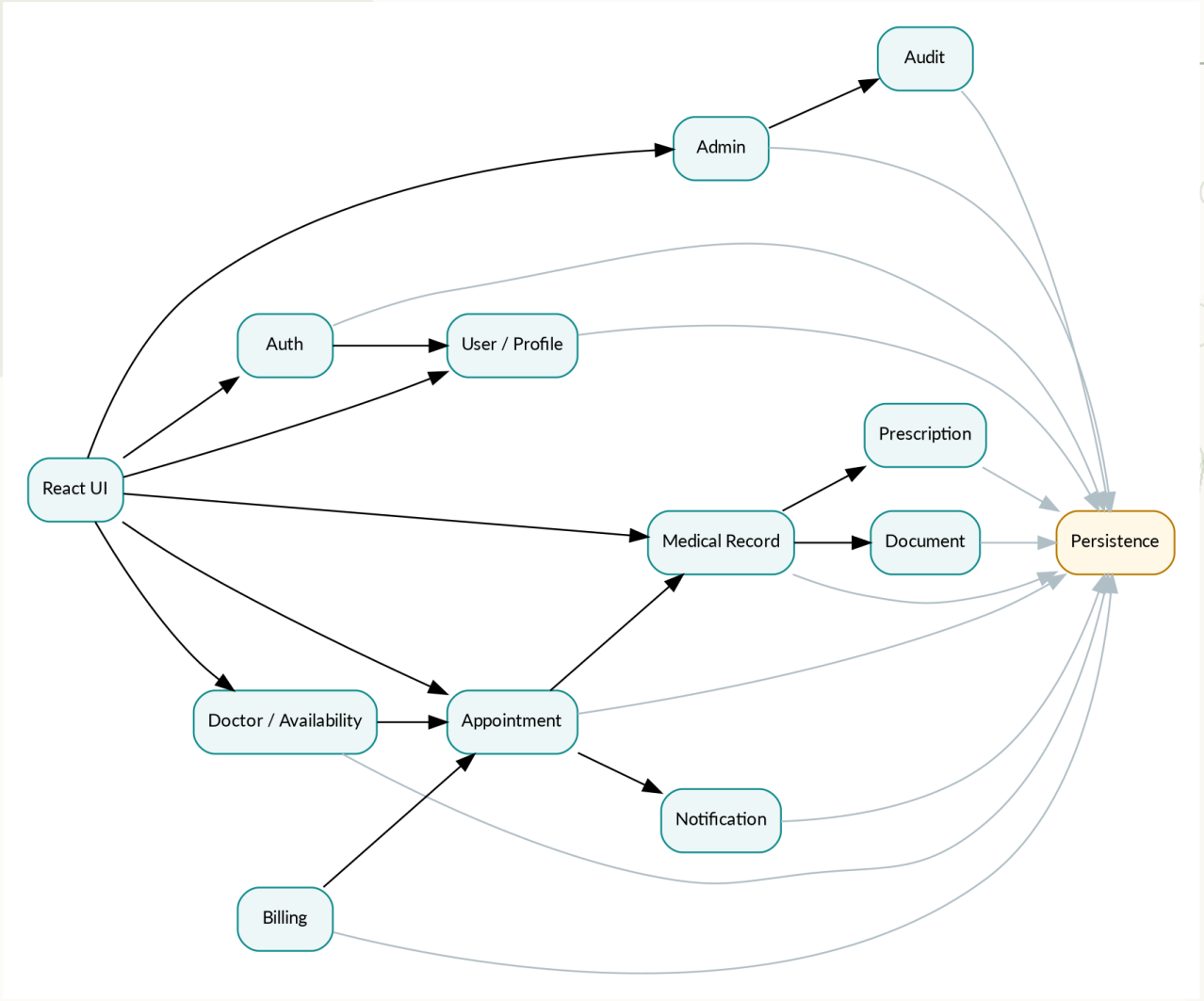


Figure 6. Major application components.

Dependency rule
Controllers call application services. Services coordinate domain rules and repositories. Modules communicate through explicit service interfaces or domain events, not direct table manipulation.

39 Security Architecture

Authentication, authorisation, and data-protection controls

Control	Implementation baseline	Risk addressed
Password storage	BCrypt with a strong work factor.	Credential disclosure.
Access token	Short-lived signed JWT.	Long-lived bearer-token exposure.
Refresh token	Rotated token in HttpOnly SameSite cookie; SHA-256 hash stored server-side.	Token theft and replay.
Authorisation	Spring Security plus service-level ownership checks.	Horizontal and vertical privilege escalation.
Input validation	DTO validation and server-side business checks.	Malformed data and unsafe state changes.
Audit	Actor, action, target, time, and outcome.	Lack of accountability.
Transport	HTTPS in production and secure cookie settings.	Network interception.
Secrets	Environment or managed secret store, never committed.	Source-code secret leakage.

Security truth
JWT does not replace server-side checks. Every protected operation still verifies role, ownership, account status, and current database state.

40 Deployment Model

Containers, proxying, persistence, and external services

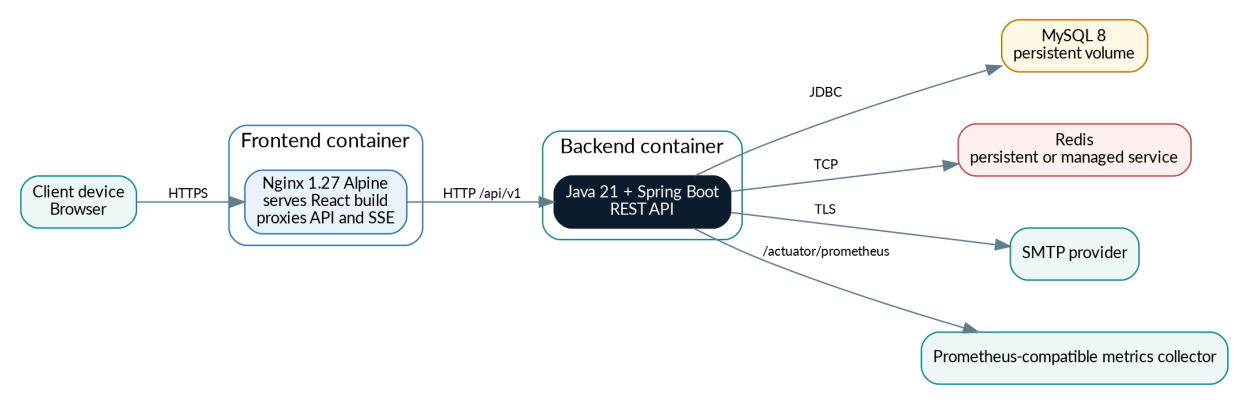


Figure 7. Container-oriented deployment model.

- Frontend Docker build uses Node 22 Alpine and an Nginx 1.27 Alpine runtime.
- Nginx serves the single-page application and proxies /api/ and /api/v1/notifications/stream.
- The SSE proxy keeps a longer read timeout to avoid premature connection closure.
- Backend, MySQL, and Redis can run through Docker Compose locally or managed services in deployment.

41 Backend Module Structure

A repeatable package pattern for each domain

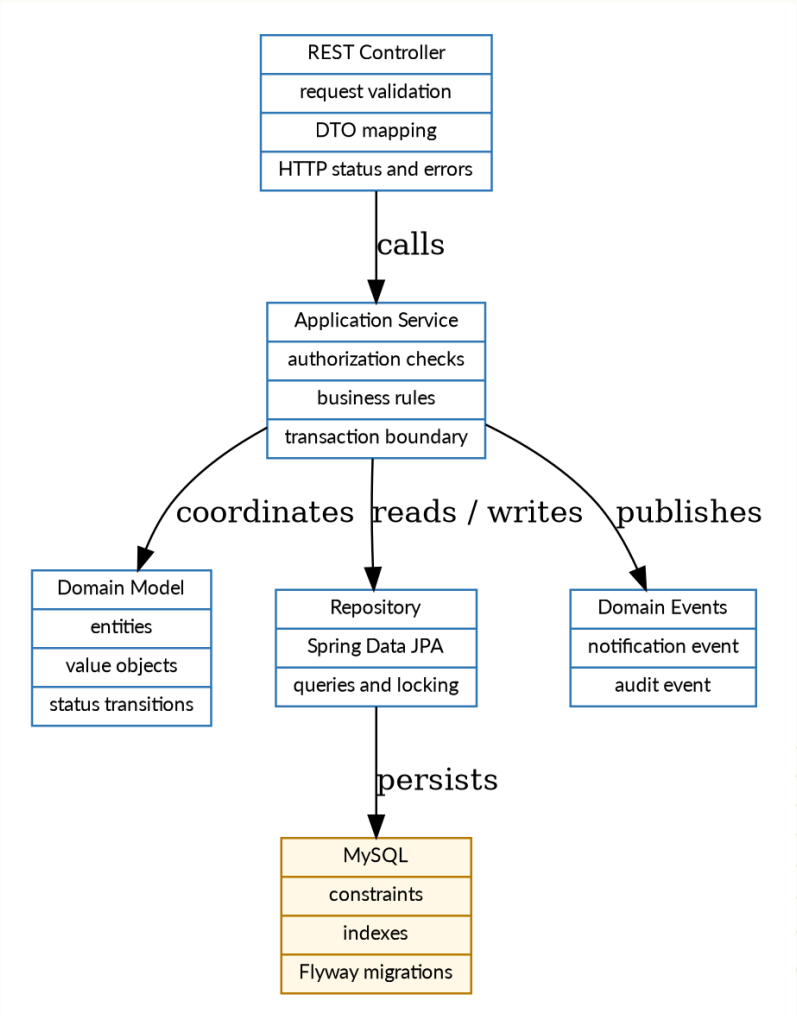


Figure 8. Standard module layering.

Layer	Responsibility
Controller	HTTP mapping, request parsing, DTO validation, response status.
Service	Use-case orchestration, authorisation, transactions, and domain rules.
Domain	Entities, value objects, enums, and valid status transitions.
Repository	Persistence queries, locks, and pagination.
Event / notification adapter	Audit and user-notification side effects.

42 Database Overview

Relational model for consistency and traceability

- UUID identifiers reduce accidental information leakage from sequential ids and support distributed creation patterns.
- Foreign keys preserve relationships between users, profiles, appointments, records, and events.
- Unique constraints and indexes support account uniqueness and booking conflict prevention.
- Optimistic locking protects editable aggregate state from silent overwrites.
- Flyway migrations keep schema changes ordered and reproducible.
- Audit and notification records are append-oriented.

Data design principle

Clinical, scheduling, billing, and security data are separated into focused tables rather than a single oversized user or appointment table.

43 Entity Relationship Diagram

Core Version 2.0 persistence model

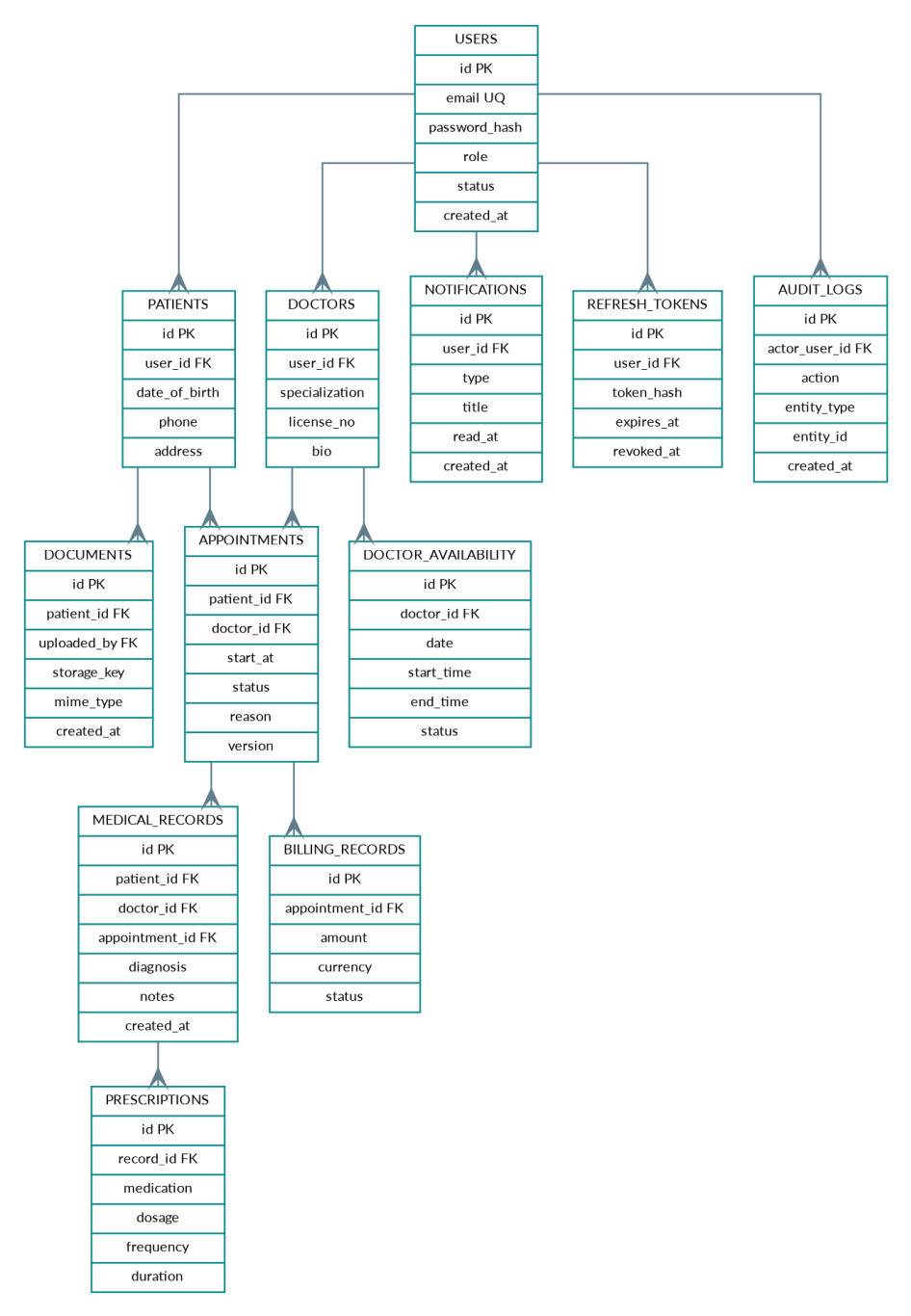


Figure 9. Logical entity relationship diagram.

Diagram scope
The ERD shows the core logical model. Exact column names and optional fields should follow the live Flyway migrations and OpenAPI schema.

44 Data Dictionary - Identity and Scheduling

Key tables and important constraints

Table	Purpose	Important constraints
users	Authentication identity and global account state.	Unique normalised email; role and status enums; password hash required.
patients	Patient-specific profile.	One-to-one with user; user role must be PATIENT.
doctors	Doctor-specific professional profile.	One-to-one with user; unique licence number when provided.
doctor_availability	Bookable time windows.	Valid time range; no duplicate or overlapping active windows.

Table	Purpose	Important constraints
appointments	Patient-doctor booking lifecycle.	Patient and doctor required; valid status transition; conflict prevention; version column.
refresh_tokens	Revocable session continuation.	Hashed token unique; expiry and revocation timestamps.

45 Data Dictionary - Care and Operations

Records, notifications, billing, documents, and audit

Table	Purpose	Important constraints
medical_records	Clinical notes linked to patient, doctor, and optionally appointment.	Author and patient required; timestamps immutable.
prescriptions	Medication instructions linked to a record.	Record and author required; validated dosage fields.
billing_records	Charges and status linked to an appointment.	Non-negative amount; ISO-style currency code.
documents	File metadata and storage reference.	Generated storage key; allowed MIME type; owner context.
notifications	Persistent user inbox.	Recipient required; read_at nullable; event type controlled.
audit_logs	Append-only security and business event trail.	Actor, action, target, and timestamp; normal APIs do not update records.

46 API Design Conventions

Stable behavior across REST endpoints

Convention	Decision
Base path	/api/v1
Authentication	Access token on protected requests; refresh token in HttpOnly cookie.
Resource naming	Plural nouns and stable identifiers.
Validation	400 for malformed or invalid input with field errors.
Authentication failure	401 for missing or invalid credentials.
Authorisation failure	403 for authenticated but forbidden access.
Missing resource	404 without exposing hidden resources.
Conflict	409 for duplicate, stale version, invalid transition, or occupied slot.
Error body	Problem-details style object with code, message, path, timestamp, and correlation id.
Pagination	page, size, sort with enforced limits.
Idempotency	Used where duplicate submission could create financial or booking side effects.

Source of truth

The generated OpenAPI document should be treated as the exact runtime contract. This catalogue is the design baseline.

47 API Catalogue - Identity and Appointments

Representative Version 2.0 endpoints

Method	Path	Purpose
POST	/api/v1/auth/register	Create a patient account.

Method	Path	Purpose
POST	/api/v1/auth/login	Authenticate and issue session credentials.
POST	/api/v1/auth/refresh	Rotate refresh credentials.
POST	/api/v1/auth/logout	Revoke refresh token and clear cookie.
GET	/api/v1/users/me	Return the current user profile.
PUT	/api/v1/users/me	Update permitted profile fields.
GET	/api/v1/doctors	Search active doctors.
GET	/api/v1/doctors/{id}/availability	Return available slots for a date range.
POST	/api/v1/doctors/me/availability	Create a doctor availability window.
POST	/api/v1/appointments	Book an appointment.
PATCH	/api/v1/appointments/{id}/reschedule	Move an appointment to a new slot.
PATCH	/api/v1/appointments/{id}/status	Apply an authorised status transition.
DELETE	/api/v1/appointments/{id}	Cancel an appointment under policy.

48 API Catalogue - Care, Notifications, and Admin

Representative Version 2.0 endpoints

Method	Path	Purpose
GET	/api/v1/patients/me/appointments	Patient appointment history.
GET	/api/v1/patients/{id}/medical-records	Authorised medical history.
POST	/api/v1/patients/{id}/medical-records	Create a medical record.
POST	/api/v1/medical-records/{id}/prescriptions	Create prescription items.
GET	/api/v1/billing/me	Patient billing summary.
POST	/api/v1/documents	Upload document metadata and file.
GET	/api/v1/documents/{id}	Download an authorised document.
GET	/api/v1/notifications	Return notification inbox.
PATCH	/api/v1/notifications/{id}/read	Mark a notification as read.
GET	/api/v1/notifications/stream	Open authenticated SSE stream.
GET	/api/v1/admin/users	List and filter users.
PATCH	/api/v1/admin/users/{id}/status	Change account status.
GET	/api/v1/admin/audit-logs	Search audit events.
GET	/actuator/health	Application health.
GET	/actuator/prometheus	Prometheus-compatible metrics.

49 Login Sequence

Secure credential verification and token creation

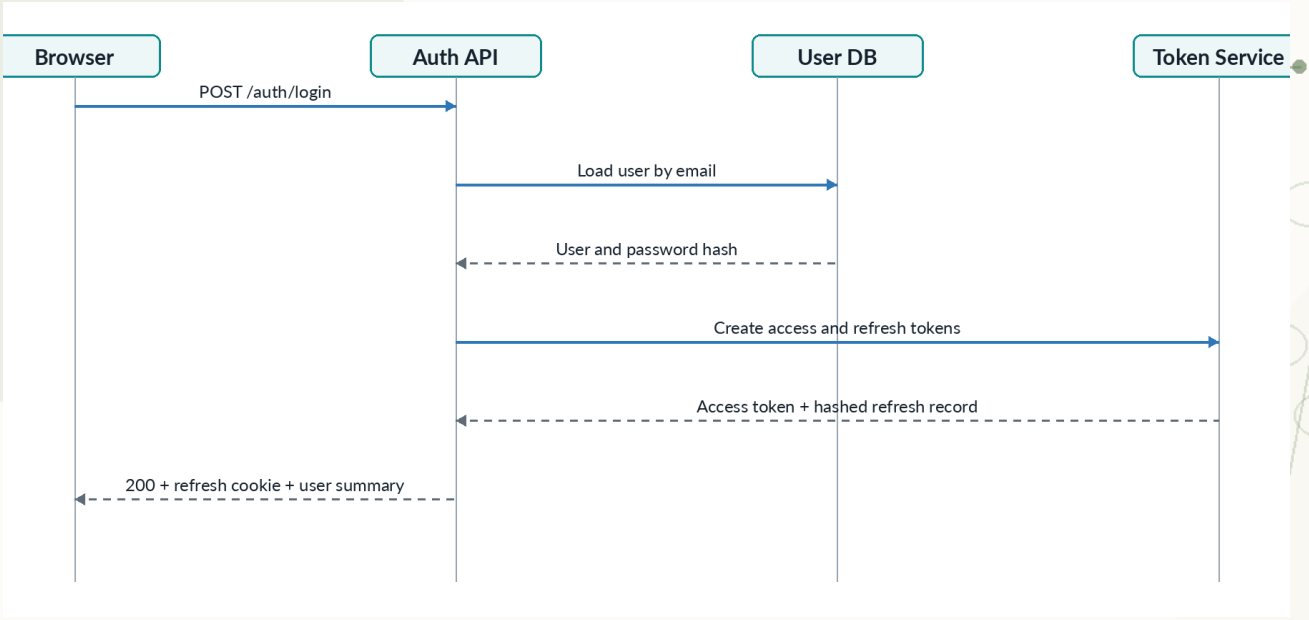


Figure 10. Login sequence.

Correction

The database verifies user state and returns a password hash. It does not create a session by itself, as suggested by the old sequence diagram.

50 Appointment Booking Sequence

Transactional conflict check and notification

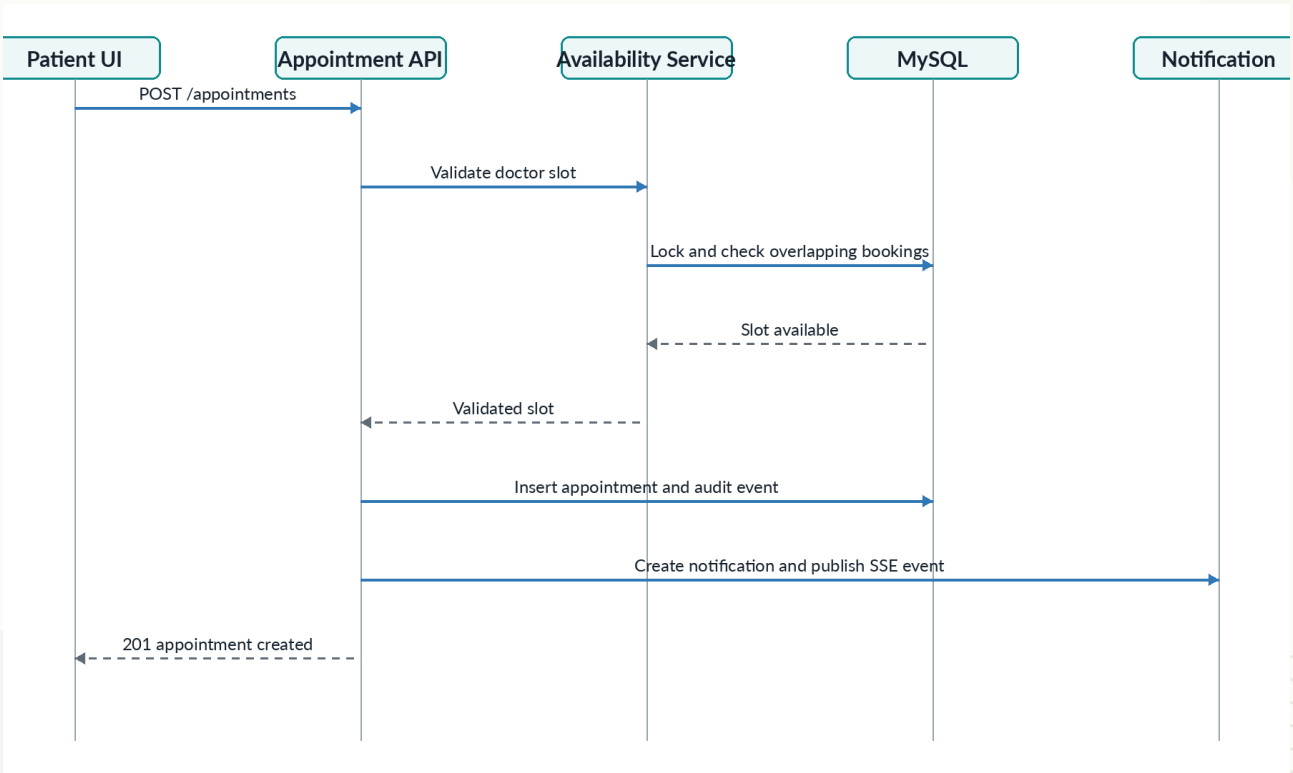


Figure 11. Appointment booking sequence.

Key safeguard

The conflict check and insert belong to one transaction so that two concurrent clients cannot both win the same slot.

51 Refresh Token Sequence

Rotation and revocation of session continuation

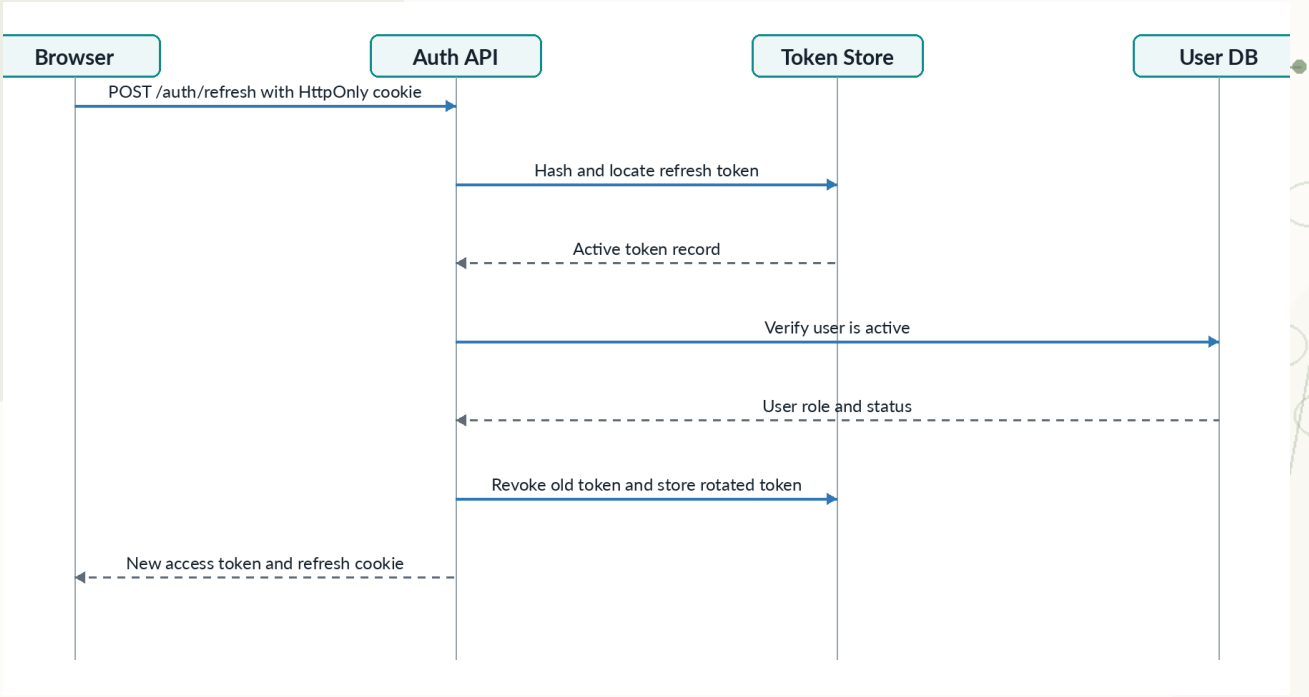


Figure 12. Refresh token rotation sequence.

Token handling
Only a hash of the refresh token is stored. The browser cookie is HttpOnly and is rotated after successful use.

52 Real-Time Notification Sequence

Persistent inbox first, SSE delivery second

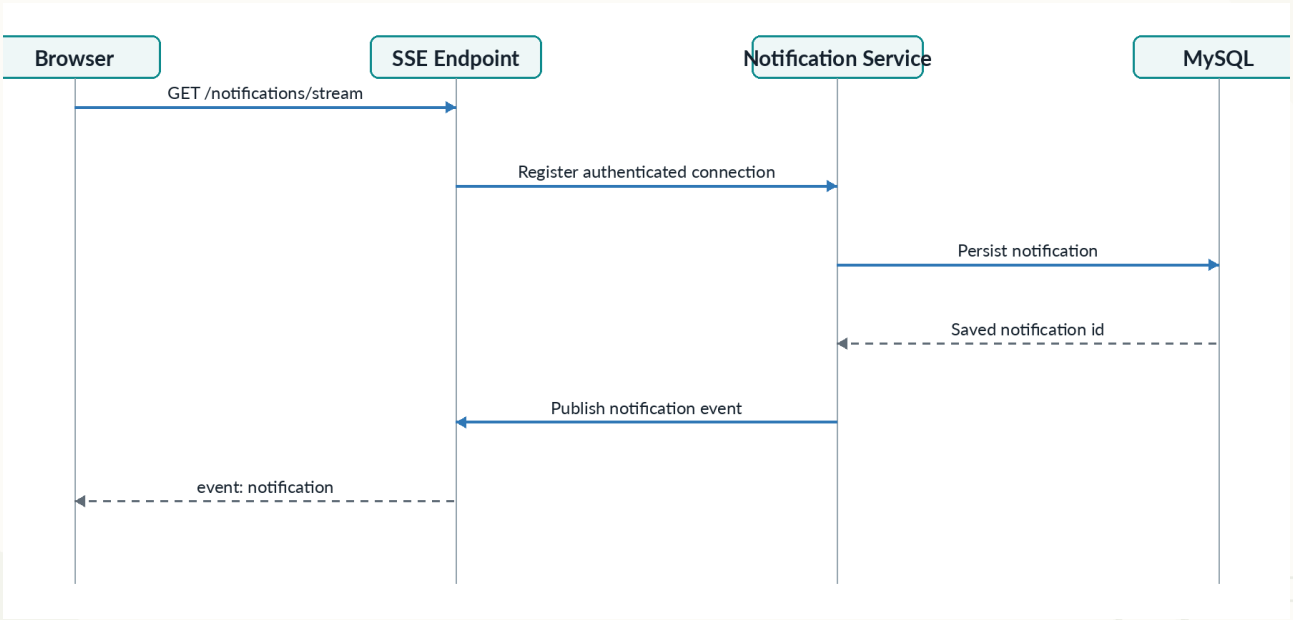


Figure 13. Notification creation and SSE delivery.

Reliability rule
A lost SSE connection does not lose the message because the notification is persisted before the event is emitted.

53 Frontend Architecture

React and TypeScript client responsibilities

Area	Responsibility
Routing	Public and protected routes with role-aware navigation.

Area	Responsibility
API client	Base URL /api/v1, JSON handling, credentials included for refresh cookie, typed responses.
Authentication state	Current user, access-token lifecycle, refresh retry, and logout cleanup.
Feature modules	Auth, patient, doctor, appointment, medical, billing, document, notification, and admin screens.
Forms	Client validation for usability, with server errors mapped back to fields.
Real-time	One authenticated SSE connection with reconnect and cleanup.
UI quality	Responsive layout, loading states, empty states, accessible labels, and stable error messages.

Frontend boundary
The browser improves usability but does not decide permissions, appointment conflicts, or valid status transitions.

54 Backend Architecture

Spring Boot responsibilities and module conventions

Concern	Implementation approach
Controllers	Thin HTTP adapters using request and response DTOs.
Services	Business rules, ownership checks, transactions, and orchestration.
Repositories	Spring Data JPA queries, pagination, locking, and persistence.
Entities	UUID ids, auditing timestamps, enums, relationships, and optimistic version.
Security	Filter chain, JWT validation, method or service authorisation, safe error handling.
Validation	Bean validation plus domain-specific checks.
Migrations	Flyway versioned SQL.
Documentation	Spring OpenAPI / Swagger UI.
Operations	Actuator health and metrics.

55 Data Consistency and Transactions

Rules that prevent silent corruption

- Registration creates the user and role profile in one transaction.
- Appointment booking checks and creates the booking in one transaction.
- Unique constraints and lock-aware queries are used where a race condition can occur.
- Status changes validate both the current state and requested next state.
- Optimistic locking detects stale updates instead of silently overwriting them.
- Notifications and audit events are created as part of, or immediately after, the successful business transaction.
- External side effects such as email are retried independently so that email failure does not roll back a valid appointment.

Practical rule
Database constraints are the final safety net. Frontend checks alone are never enough for concurrency.

56 Testing Strategy

Layered tests focused on the highest-risk behavior

Test level	Purpose	Examples
Unit	Verify isolated business rules.	Status transitions, token expiry, appointment policy, DTO mapping.

Test level	Purpose	Examples
Repository	Verify queries and constraints.	Overlap search, pagination, unique email, optimistic version.
Service integration	Verify transactions and authorisation.	Booking conflict, record access, token rotation, audit creation.
Controller / API	Verify HTTP contract.	Validation, status codes, error body, role restrictions.
Security	Verify abuse cases.	Horizontal access, expired token, revoked refresh, role escalation.
Frontend	Verify user flows and state.	Form errors, protected routes, refresh retry, SSE updates.
Deployment smoke	Verify packaged system.	Containers start, migrations run, health is UP, login succeeds.

57 Test Matrix

Minimum verification set before publishing a release

ID	Scenario	Expected result
AUTH-01	Valid login	200; refresh cookie; user summary.
AUTH-02	Wrong password	401; no token; generic message.
AUTH-03	Refresh rotation	Old token revoked; new token issued.
AUTH-04	Revoked refresh reuse	401 and security event.
RBAC-01	Patient calls admin API	403.
RBAC-02	Doctor reads unrelated patient	403 or safe 404.
APT-01	Book free slot	201 pending appointment.
APT-02	Concurrent same-slot booking	One succeeds; one returns 409.
APT-03	Invalid status transition	409; no data change.
MED-01	Doctor creates record	201; author and patient linked.
DOC-01	Disallowed file type	400; no metadata remains.
SSE-01	Notification event	Inbox record exists and connected user receives event.
OPS-01	Fresh database startup	Flyway completes and health is UP.
ERR-01	Unexpected service exception	Safe 500 body with correlation id; no stack trace exposed.

58 CI/CD and Deployment

Repeatable build and release workflow

105. Run backend compilation, formatting checks, and automated tests.
106. Run frontend type checking, linting, and production build.
107. Build backend and frontend container images.
108. Start dependent services for integration or smoke tests.
109. Apply Flyway migrations in a controlled deployment step.
110. Deploy backend and frontend containers with environment-specific secrets.
111. Verify health, login, core appointment flow, and SSE connection.
112. Record the release version and rollback point.

Release rule

A successful image build is not enough. The release is accepted only after migrations, health checks, and core smoke tests pass.

59 Observability and Monitoring

Health, metrics, logs, and audit are different signals

Signal	Purpose	Examples
Application health	Determine whether dependencies are ready.	Database, Redis, disk, mail adapter readiness.
Metrics	Observe performance and volume.	Request count, latency, error rate, JVM, connection pool.
Structured logs	Diagnose technical failures.	Correlation id, path, status, exception category.
Audit logs	Track important user and admin actions.	Login, appointment change, record access, account suspension.
Alerts	Notify operators about abnormal conditions.	Health failure, rising 5xx rate, exhausted connections.

Do not mix them

Audit logs are not debug logs. Sensitive medical content should not be copied into application logs.

60 Error Handling

Stable errors without exposing internals

Category	HTTP status	Example code	Client action
Validation	400	VALIDATION_ERROR	Correct highlighted fields.
Authentication	401	AUTHENTICATION_REQUIRED	Sign in or refresh session.
Authorisation	403	ACCESS_DENIED	Do not retry with same role.
Missing resource	404	RESOURCE_NOT_FOUND	Refresh list or return.
Conflict	409	APPOINTMENT_SLOT_TAKEN	Reload availability.
Stale update	409	VERSION_CONFLICT	Reload latest record.
Rate limit	429	TOO_MANY_REQUESTS	Wait before retry.
Unexpected failure	500	INTERNAL_ERROR	Show safe message and correlation id.

Error rule

The client receives a safe explanation. Technical details remain in protected logs associated with the correlation id.

61 Privacy and Security Considerations

Controls appropriate for an academic healthcare prototype

- Collect only fields needed for the documented workflows.
- Avoid placing medical content, passwords, raw tokens, or uploaded documents in logs.
- Use least-privilege database and storage credentials.
- Restrict document access using patient context and authenticated identity.
- Separate administrator operations from clinical record editing.
- Expire and revoke refresh credentials.
- Use HTTPS, secure cookies, and strict CORS in deployment.
- Back up encrypted infrastructure and test restoration.
- Document data retention and deletion policy before real deployment.

No compliance claim

The document does not claim GDPR, HIPAA, medical-device, or national healthcare certification. Those require legal, organisational, and operational controls beyond application code.

62 Limitations and Exclusions

What is intentionally not overstated

Limitation	Consequence
Private repository was not available for a line-by-line audit while producing this edition.	Exact package names and endpoint paths should be checked against the generated OpenAPI and Flyway migrations before final submission.
Academic load profile	Performance targets are engineering goals, not production benchmarks.
No external payment integration	Billing records do not prove completed financial transactions.
No certified identity verification	User identity depends on the registration and administration process.
No real hospital deployment	Usability and workflow assumptions require validation with healthcare staff.
No emergency or diagnostic automation	The system must not be used for triage or clinical decision support.

63 Roadmap

Practical next improvements in priority order

Priority	Item	Reason
P1	Repository verification	Align every endpoint, entity, and module name with live code and generated OpenAPI.
P1	Automated security tests	Add ownership, refresh reuse, rate-limit, and document-access tests.
P1	Booking constraint	Prove concurrent conflict protection with an integration test against MySQL.
P2	Email reliability	Use an outbox or retry queue for transactional messages.
P2	Document storage hardening	Signed access, malware scanning, size limits, and retention policy.
P2	Operational dashboards	Expose release, health, error, and latency information.
P3	Reporting	Add carefully scoped exports with audit and privacy controls.
P3	Additional roles	Introduce nurse or receptionist permissions only after a clear access matrix.

Best next step

Do not add AI, Kafka, or microservices before the current security tests, booking concurrency, documentation-code alignment, and deployment smoke tests are solid.

64 Migration from the Old MERN Version

What changed and why

Area	Old academic implementation	Version 2.0 rebuild
Frontend	HTML, CSS, and vanilla JavaScript.	React, TypeScript, and Vite.
Backend	Node.js and Express.	Java 21 and Spring Boot.
Security	JWT and bcrypt with mixed documentation.	Spring Security, short access token, rotating refresh cookie, BCrypt, RBAC, and audit.
Persistence	MySQL schema and direct application queries.	JPA/Hibernate, Flyway, transactions, UUIDs, and optimistic locking.

Area	Old academic implementation	Version 2.0 rebuild
Real-time updates	Notification feature described, implementation inconsistent.	Persistent notifications with SSE delivery.
Operations	Basic Docker and presentation-level deployment notes.	Docker, Nginx reverse proxy, Actuator, Prometheus-compatible metrics, and environment configuration.
Documentation	D1, D2, and D3 contained duplicate and conflicting statements.	One consolidated baseline with explicit status and exclusions.

65 Consistency Fixes Applied

Specific corrections made to the old documentation

Topic	Correction
Product name	Uses Synapse HEA consistently. HealthCore and Healthcare Appointment System are treated as old naming drift.
Technology stack	Removes Node.js and Express from the current architecture and documents the Java rebuild.
Auth model	Separates access token, refresh token, cookie, rotation, revocation, and ownership checks.
Requirement IDs	Creates one normalised FR1-FR32 catalogue instead of duplicated and mixed identifiers.
Scope	Moves urgent triage, beds, equipment, insurance claims, and AI diagnosis outside the current baseline.
Use cases	Uses one use case per user goal instead of treating quality constraints as user actions.
Diagrams	Corrects responsibility boundaries, transaction points, and alternative flows.
Implementation status	Separates current core, extended, operational, and excluded capabilities.
Authorship	Clearly identifies Mohammad Nomaan Munshi as the sole author of the redesign, implementation, and documentation.

66 Conclusion

A coherent and honest baseline for the rebuilt project

Synapse HEA Version 2.0 is documented as an independent Java and Spring Boot system by Mohammad Nomaan Munshi. The system is intentionally presented as a modular monolith with a clear security model, relational data design, transactional appointment workflow, real-time notification channel, and repeatable deployment approach.

- The current architecture is more coherent than the old MERN documentation.
- The requirement catalogue now matches the stated modules and project boundaries.
- Diagrams show realistic ownership and transaction responsibilities.
- Unsupported features are excluded instead of being presented as completed.
- The next quality step is verification against the live repository and OpenAPI output.

Portfolio wording

Independent Java/Spring Boot healthcare-management platform by Mohammad Nomaan Munshi, modernised with React, TypeScript, MySQL, Redis, Docker, Nginx, secure token rotation, audit logging, and real-time notifications.

67 Glossary

Terms used throughout the document

Term	Meaning
API	Application Programming Interface.
BCrypt	Adaptive password-hashing function.

Term	Meaning
DTO	Data Transfer Object used at API boundaries.
Flyway	Versioned database migration tool.
HEA	Healthcare Management System project name used by Synapse.
HttpOnly	Cookie attribute that prevents JavaScript access.
JWT	JSON Web Token used for short-lived access credentials.
JPA	Java Persistence API used through Hibernate.
RBAC	Role-Based Access Control.
SSE	Server-Sent Events, a one-way real-time server-to-browser stream.
UUID	Universally Unique Identifier.
Optimistic locking	Version-based detection of concurrent updates.
Modular monolith	One deployable application split into clear business modules.

A1 Appendix A - Requirement Status

Implementation and verification labels

ID	Requirement	Status	Evidence needed
FR1	User registration	Current baseline	Verify against source and tests
FR2	Login	Current baseline	Verify against source and tests
FR3	Password management	Current baseline	Verify against source and tests
FR4	Profile management	Current baseline	Verify against source and tests
FR5	Doctor search	Current baseline	Verify against source and tests
FR6	Doctor availability	Current baseline	Verify against source and tests
FR7	Appointment booking	Current baseline	Verify against source and tests
FR8	Appointment modification	Current baseline	Verify against source and tests
FR9	Appointment status workflow	Current baseline	Verify against source and tests
FR10	Medical record access	Current baseline	Verify against source and tests
FR11	Medical record maintenance	Current baseline	Verify against source and tests
FR12	Prescription management	Current baseline	Verify against source and tests
FR13	Billing records	Current baseline	Verify against source and tests
FR14	Document handling	Current baseline	Verify against source and tests
FR15	Notification inbox	Current baseline	Verify against source and tests
FR16	Real-time notifications	Current baseline	Verify against source and tests
FR17	Role-based access control	Current baseline	Verify against source and tests
FR18	Token lifecycle	Current baseline	Verify against source and tests
FR19	Audit logging	Current baseline	Verify against source and tests
FR20	Admin user management	Current baseline	Verify against source and tests
FR21	Doctor schedule management	Current baseline	Verify against source and tests
FR22	Appointment history	Current baseline	Verify against source and tests
FR23	Patient dashboard	Current baseline	Verify against source and tests
FR24	Doctor dashboard	Current baseline	Verify against source and tests
FR25	Admin dashboard	Current baseline	Verify against source and tests

ID	Requirement	Status	Evidence needed
FR26	Reporting	Extended baseline	Verify against source and tests
FR27	API documentation	Current baseline	Verify against source and tests
FR28	Health monitoring	Current baseline	Verify against source and tests
FR29	Error handling	Current baseline	Verify against source and tests
FR30	Schema migration	Current baseline	Verify against source and tests
FR31	Backup readiness	Operational requirement	Verify against source and tests
FR32	Logout and revocation	Current baseline	Verify against source and tests

A2 Appendix B - Example API Payloads

Illustrative JSON structures, not a substitute for OpenAPI

Login request

```
{
  "email": "patient@example.com",
  "password": "********"
}
```

Appointment request

```
{
  "doctorId": "<uuid>",
  "startAt": "2026-09-14T10:30:00+02:00",
  "reason": "Follow-up consultation"
}
```

Problem response

```
{
  "code": "APPOINTMENT_SLOT_TAKEN",
  "message": "The selected time is no longer available.",
  "status": 409,
  "path": "/api/v1/appointments",
  "correlationId": "<id>"
}
```

A3 Appendix C - Repository Structure

Recommended structure for the Java full-stack rebuild

```
synapse-hea-java-phase2/
  backend/
    src/main/java/.../
      auth/ patient/ doctor/ appointment/
      medical/ prescription/ billing/ document/
      notification/ admin/ audit/ common/
    src/main/resources/
      db/migration/
      application.yml
    src/test/java/.../
    pom.xml
    Dockerfile
  frontend/
    src/
      api/ auth/ components/ features/ pages/ routes/
    package.json
    vite.config.ts
    nginx.conf
    Dockerfile
  docker-compose.yml
  .github/workflows/
  docs/
  README.md
```

Repository rule

Generated files, secrets, uploads, and local database data must not be committed. Environment templates should contain names only, never real credentials.